Supplementary Material: Automated Inference and Testing of Separation Logic Specifications with Magic Wands

ANONYMOUS AUTHOR(S)

This document is the supplementary material of the paper. It contains the doubly linked list predicates, the prompt templates, the step-by-step workflow of the testing algorithm on the running example, the full grammar of the CN fragment, the execution of the running example under Fulminate, and the benchmarks with the additional experimental results.

A Doubly Linked List Predicates

This appendix shows the doubly linked list that the three doubly linked list benchmarks (Table S2) operate on, and the predicates that are needed to state the loop invariants over it. Fig. S1 is a doubly linked list whose nodes have `prev`, `value` and `next` fields. The pointer `h` points to the head of the list and `haddr` to the node that the loop has reached so far. The nodes before `haddr` (green) have been traversed, and the nodes from `haddr` on (yellow) have not.

Compared to the singly linked list, the predicate $dllrep(l, h, preva)$ of Fig. S2 has one more argument. A node does not only point to its successor, it also points back to its predecessor, so the predicate has to carry the address of the previous node: the cell at h holds $(hd, preva, ht)$, and the predicate is called recursively on the tail with h as the new previous address. The length of the list and the values it holds are given by the mathematical list l, as for the singly linked list.

The left side of Fig. S2 is the invariant that has to be written without the magic wand, only with the separating conjunction. Three things are needed in addition to $dllrep$. First, a second recursive predicate $dllseg(l, h, preva, enda)$ for the part of the list that has been traversed: a segment that starts at h and ends at the address $enda$ of the current node, instead of at nil, and that has no counterpart in the precondition or the postcondition. Second, ghost variables for the two halves of the list, l_1 and l_2, and for the addresses on the boundary between them, l_{1end} and l_{2beg}. Third, a pure fact $l = l_1 +\!\!+ l_2$ that ties the two halves of the list. The right side is the that the wand elegantly captures. It needs no segment predicate, no concatenation fact and no boundary address of the traversed part: the ghost variables l_1, l_{1beg} and l_{1prev} describe the part of the list that remains, $dllrep(l_1, l_{1beg}, l_{1prev})$ is the resource that the loop owns, and the wand states that giving up this part returns the whole list $dllrep(l, p, nullval)$. The traversed part is thus never described explicitly, and the base predicate $dllrep$ is the only predicate that the inference has to use.

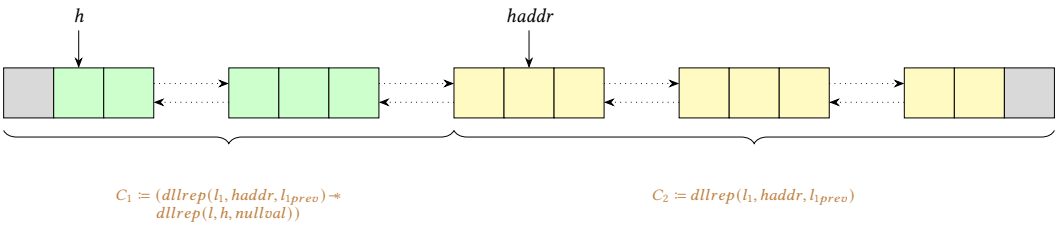


Fig. S1. A doubly linked list, with each node holding `prev`, `value`, and `next` fields. `h` points to the head of the list and `haddr` points to a node in the middle, the boundary reached so far by the loop. The loop invariant is $C_1 * C_2$.

2026. ACM XXXX-XXXX/2026/10-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

$(l : list\ Z,\ h, preva, enda : address)$
$dllseg(l, h, preva, enda) \coloneqq$
$\quad l = [\,] \rightarrow (h = enda), emp$
$\quad l = hd :: tl \rightarrow minI32 \le hd \le maxI32,$
$\exists ht,\ h \mapsto (hd, preva, ht) * dllseg(tl, ht, h, enda)$

$\exists l_1 : list\ Z, \exists l_{1end} : val,\ \exists l_2 : list\ Z,\ \exists l_{2beg} : val,$
$l = l_1 +\!+ l_2,$
$dllseg(l_1, p, nullval, l_{2beg}) * dllrep(l_2, l_{2beg}, l_{1end})$

$(l : list\ Z,\ h, preva : address)$
$dllrep(l, h, preva) \coloneqq$
$\quad l = [\,] \rightarrow (h = nil), emp$
$\quad l = hd :: tl \rightarrow minI32 \le hd \le maxI32,$
$\exists ht,\ h \mapsto (hd, preva, ht) * dllrep(tl, ht, h)$

$\exists l_1 : list\ Z,\ \exists l_{1beg} : val,\ \exists l_{1prev} : val,$
$dllrep(l_1, l_{1beg}, l_{1prev}) *$
$(dllrep(l_1, l_{1beg}, l_{1prev}) \mathrel{-\!\!*} dllrep(l, p, nullval))$

Fig. S2. Comparison of two loop-invariant formulations for the doubly-linked list, defined by `struct dll { int key; struct dll *prev, *next; };` : a conjunction-based invariant using the extra `dllseg` predicate (left) versus a magic-wand-based invariant using only the base `dllrep` predicate (right)

B Prompt Template

The inference LLM is prompted in two stages. Fig. S3 is the initial prompt. It first states the task: the invariant is written in CN as a sequence of `take` clauses and pure facts, followed by at most one wand with an antecedent and a consequent; every variable that is bound by a `take` inside a wand has to be pinned by an equation, and only the boolean functions that are already in the file may be called. The prompt then fixes the format of the answer to exactly three blocks, `NEWPRED` for new predicates, `NEWFUNC` for new functions and `INV` for the invariant. A few worked examples of other benchmarks follow (elided in the figure). The prompt ends with the instance to be solved: the benchmark file, the loop and its line, the template with the ghost variables and the pure facts that the invariant has to use, and the C file with its contract, its predicates and the template.

Infer the loop invariant of the loop under test, written in CN. The file already has a complete requires/ensures contract and all predicates and functions needed to state it. An invariant is a sequence of take clauses and pure facts, followed by a set of wands with an antecedent and a consequent. Every take-bound variable inside a wand must be pinned by an equation. Only call boolean functions that are already given. Return exactly three blocks and nothing else: NEWPRED (new predicates), NEWFUNC (new functions) and INV (the invariant).

Here are a couple of examples of how invariants look for given files: [addFromN, copyList, ..] (elided for space)

Please infer the invariant as asked below:
Benchmark File: ⟨**File**⟩
Loop: ⟨**i**⟩ **of** ⟨**n**⟩ Line: ⟨**Line**⟩
Template: ⟨**Ghost variables, pure facts**⟩
File Contents: ⟨**C file with contract, predicates, template**⟩

Fig. S3. The initial prompt template for the inference LLM

When an invariant is rejected, the LLM is re-promted with one of the templates of Fig. S4. The first line names the kind of rejection: a violation of the template (a ghost variable or a propositional fact of the template that the invariant does not use, a new recursive predicate, or a wand that is not in normal form), a compilation error, or a failure of the test. For a failed test, the message gives the category of the check that failed (`ENTRY`, `PRESERVATION`, `EXIT`, `DEFINITION` or `GENERATOR`), the test case on which it failed, and the semantic gap: the conjunct of the invariant, the value it evaluated to and the value it needed to have. The message ends with the list of the invariants that have already been rejected, which the LLM must not submit again.

The invariant you wrote was rejected: ⟨**Template** Compile Test Failure⟩. Please re-generate the invariant following the instructions and error message history.

Your invariant failed testing. Category: ⟨**ENTRY, PRESERVATION, EXIT, DEFINITION, or GENERATOR**⟩. Failed test case: ⟨**Input**⟩. Semantic gap: the conjunct ⟨**Conjunct**⟩ evaluated to ⟨**Actual**⟩ but needed to be ⟨**Expected**⟩. Invariants already tried and rejected: ⟨**History**⟩. Do not resubmit any of these; diagnose the mistake and fix it.

Fig. S4. Templates used to re-prompt the inference LLM

C Workflow: Testing the Loop Invariant of `addOne`

This appendix is the step-by-step workflow of the testing algorithm on the running example. Algorithms 1 to 3 and Figure 1 are those of the main paper.

We walk Algorithm 2 and Algorithm 3 through the function of Figure 1. Let $f = \lambda\, x \Rightarrow x{+}1$. Here l is the list that the *Pre* gives at head h, and let ls be the list at `curr`. The shape invariant that the template of Figure 1 asks for is

$$C_2 * C_1, \qquad C_2 \coloneqq lrep(ls, curr), \qquad C_1 \coloneqq lrep(map\ f\ ls, curr) \mathrel{-\!\!*} lrep(map\ f\ l, h).$$

In the notation of Algorithm 2, $inv = C_2 * C_1$, where C_2 is wand-free and C_1 is a wand, so $A_{wa} = lrep(map\ f\ ls, curr)$ and $A_{wb} = lrep(map\ f\ l, h)$.

The test case. Take $l = [1, 2, 3]$, stored in three nodes at addresses $a_1 = h$, a_2 and a_3, and consider the checkpoint reached after one iteration of the loop, so $curr = a_2$ and $ls = [2, 3]$. The test heap is

$$\sigma_i = \{\, a_1 \mapsto (2, a_2),\ a_2 \mapsto (2, a_3),\ a_3 \mapsto (3, nil) \,\},$$

where the first cell already holds the incremented key. We identify a layout with the set of addresses of its cells.

Algorithm 2, the star part. For $a = C_2 = lrep(ls, curr)$, Consume reads the list at a_2 from σ_i. It reads the cells a_2 and a_3, binds $ls = [2, 3]$, and finds no clash, so $\sigma_c = \{a_2, a_3\}$. The cell a_1 is still unaccounted for.

Algorithm 2, the wand part. The component C_1 is a wand, so the algorithm calls Algorithm 3.

(1) Postulate builds the antecedent (i.e., temporary heap without taking real ownership). The addresses and next pointers come from σ_i at `curr`, and the values come from A_{wa}, which says the suffix holds $map\ f\ ls = [3, 4]$. This gives $\sigma_{wa} = \{a_2 \mapsto (3, a_3),\ a_3 \mapsto (4, nil)\}$, a layout over $\{a_2, a_3\}$, and there is no clash. It also registers a borrow: stop at address $curr = a_2$ and substitute the value $[3, 4]$.

(2) BorrowInner constructs $A_{wb} = lrep(map\ f\ l, h)$ by walking the list from h. The cell a_1 is read from σ_i, giving key 2 and next a_2. The next step reaches $a_2 = curr$, where the borrow fires: the walk returns $[3, 4]$ and does not read a_2 and a_3 of σ_i. The value built is $Cons(2, [3, 4]) = [2, 3, 4]$, which is $map\ f\ l$, so the consequent's equation holds. The borrow was consumed. The cells $\{a_2, a_3\}$ of the consequent were supplied by σ_{wa}, which is σ_{wb}.

(3) BorrowOuter takes the part of the consequent's layout that σ_{wa} did not supply from σ_i. That is the cell a_1, so $\sigma_{wf} = \{a_1\}$, and Wand returns it.

Back in Algorithm 2, $\sigma_{wf} \cap (\sigma_c \uplus \sigma_w) = \{a_1\} \cap \{a_2, a_3\} = \emptyset$, so $\sigma_w = \{a_1\}$. Finally $\sigma_c \uplus \sigma_w = \{a_1, a_2, a_3\} = \sigma_i$, so the algorithm returns $\emptyset$: the invariant holds at this checkpoint for σ_i, and Algorithm 2 moves to the next test. The footprint $\{a_1 \mapsto (2, a_2)\}$ is what the wand owns: combined with any list $[3, 4]$ at a_2 it gives $[2, 3, 4] = map\ f\ l$ at h.

Other checkpoints and a wrong invariant. The same σ_i is checked at every checkpoint, and the wand footprint grows as `curr` advances. At loop entry $curr = h$, so the borrow fires at the first step of the walk and $\sigma_{wf} = \emptyset$. At loop exit $curr = nil$, $\sigma_c = \emptyset$, the consequent walks all three cells, and $\sigma_{wf} = \{a_1, a_2, a_3\} = \sigma_i$. Now suppose the wand's antecedent were $lrep(ls, curr)$ instead of $lrep(map\ f\ ls, curr)$, that is, the wand claimed that the suffix comes back unchanged. Postulate would then store the value $ls = [2, 3]$ at a_2, and the walk from h would build $Cons(2, [2, 3]) = [2, 2, 3]$, which differs from $map\ f\ l = [2, 3, 4]$. BorrowInner would fail, Algorithm 3 would return fail, and Algorithm 2 would return σ_i to Algorithm 1 as the failed test case.

D CN Syntax

Fig. S5 is the complete grammar of the CN fragment that Smeagol reads and writes, with the additions for wands in purple. A program is a sequence of top-level definitions and C functions. A C function carries a specification (*function_spec*) with a `requires` part and an `ensures` part, and each of its loops carries an invariant introduced by `inv`. Both are sequences of conditions (*spec*): a `take` binds the value that a resource predicate returns and claims ownership of the memory it covers, an `assert` or a plain boolean term is a constraint, a `let` introduces a local name, and `return` ends a predicate with its value. The only construct that we add is `wand`($spec_a$, $spec_b$), whose two arguments are sequences of conditions of the form `take` *id* = *spec*; *spec**; *id* == *t*; *spec**, where $*$ stands for zero or more conditions: each take of a wand is pinned by an equation on the identifier that it binds.

Fig. S6 shows the definitions that these constructs are used with, for the `addOne` function of the main paper. The C structure `sllist` is the node of the list. The datatype `List` is the mathematical list, with a constructor `Nil` for the empty list and a constructor `Cons` that holds a 32-bit integer head and the tail. The predicate `Lrep`, marked `[rec]` because it is recursive, relates the memory to the mathematical list: if the pointer `p` is null it returns `Nil`; otherwise it takes ownership of the cell at `p` with `RW<struct sllist>`, which binds `H` to the contents of the cell, asserts that the head is a 32-bit integer, takes the rest of the list with a recursive call on `H.tail`, and returns a `Cons` of the head and the tail. The functions `AddOne` and `allSafeToIncrement` are pure functions on the mathematical list, written with `match`: `AddOne` adds one to every element, and `allSafeToIncrement` is a boolean function that holds when no element overflows if it is incremented. The latter appears as a propositional fact in the contract and in the template of the loop, for instance as `allSafeToIncrement(Lhead)` in the `requires` part of `addOne`. Together, they give the invariant of the main paper, `take` *Lcurr* = `Lrep`(*curr*) followed by `wand(take` *Lcurr1* = `Lrep`(*curr*); ...), whose antecedent and consequent each take a list and pin it with an equation over `AddOne`.

$$
\begin{array}{ll}
\textit{toplevel_def} & \coloneqq \textit{c_function_def} \mid \textit{spec_predicate_def} \mid \textit{spec_function_def} \mid \textit{datatype_def} \\
\textit{c_function_def} & \coloneqq \textit{cty}\ f(\textit{cty}_1\ \textit{id}_1, \ldots, \textit{cty}_n\ \textit{id}_n)\ /{*}@\ \texttt{requires}\ \textit{conditions}\ \texttt{ensures}\ \textit{conditions}'\ @*\ /\ \textit{block} \\
\textit{loop} & \coloneqq \texttt{while}\ (e)\ /{*}@\ \texttt{inv}\ \textit{conditions}\ @*\ /\ \textit{block} \\
\textit{condition} & \coloneqq \texttt{take}\ \textit{id} = \textit{resource} \mid \textit{constraint} \mid \texttt{let}\ \textit{id} = t \mid \texttt{wand}(\textit{conditions}, \textit{conditions}); \\
\textit{conditions} & \coloneqq \textit{condition}_1;\ \ldots;\ \textit{condition}_n; \\
\textit{resource} & \coloneqq p(t_1, \ldots, t_n) \qquad \textit{constraint} \coloneqq t \\
\textit{function_spec} & \coloneqq /{*}@\ \texttt{requires}\ \textit{spec}_1\ \texttt{ensures}\ \textit{ret}.\textit{spec}_2\ @*\ / \\
\textit{spec} & \coloneqq \texttt{return}\ t \mid p(t_1, \ldots, t_i) \mid \texttt{assert}\,(t);\ \textit{spec} \mid \texttt{let}\ \textit{id} = t;\ \textit{spec} \mid \texttt{take}\ \textit{id} = \textit{spec};\ \textit{spec}' \\
 & \quad \mid \texttt{if}\,(t)\,\{\textit{spec}_1\}\ \texttt{else}\ \{\textit{spec}_2\} \mid \texttt{wand}(\textit{spec}_a, \textit{spec}_b);\ \textit{spec} \mid \cdot \\
\textit{spec}_a,\ \textit{spec}_b & \coloneqq \texttt{take}\ \textit{id} = \textit{spec};\ \textit{spec}^*;\ \textit{id} == t;\ \textit{spec}^* \\
p & \coloneqq \texttt{RW}\langle \textit{cty} \rangle \mid \textit{id} \\
\textit{spec_function_def} & \coloneqq \texttt{function}\ \textit{bty}\ f(\textit{bty}_1\ \textit{id}_1, \ldots, \textit{bty}_k\ \textit{id}_k)\ \{t\} \\
\textit{spec_predicate_def} & \coloneqq \texttt{predicate}\ \textit{bty}\ p(\textit{bty}_1\ \textit{id}_1, \ldots, \textit{bty}_k\ \textit{id}_k)\ \{\textit{spec}\} \\
\textit{spec_term},\ t & \coloneqq v \mid \{t\} \mid \textit{id} \mid \textit{suop}\ t \mid t_1\ \textit{sbop}\ t_2 \mid \texttt{if}\,(t)\ t_1\ \texttt{else}\ t_2 \mid f(t_1, \ldots, t_i) \\
\textit{datatype_def} & \coloneqq \texttt{datatype}\ D\ \{C_1\{\textit{bty}\ f_1, \ldots\}, \ldots\} \\
\textit{bty} & \coloneqq \texttt{i32} \mid \texttt{u32} \mid \texttt{boolean} \mid \texttt{pointer} \mid \texttt{datatype}\ D \\
\textit{cty},\ T & \coloneqq \texttt{i32} \mid \texttt{u32} \mid \texttt{sll} \mid \texttt{dll} \mid \texttt{tree} \mid \textit{cty}*
\end{array}
\tag{1}
$$

Fig. S5. The CN fragment used for specifications and loop invariants, in the style of Fulminate's §2.4 (the definitions down to *resource*) and §3.1 (from *function_spec*), with wand added to both. p ranges over resource predicates (`Lrep`, `Trep`, and so on, see the grammar of the assertions in the main paper) and `RW`, T over the cell types of the benchmarks (`sll`, `dll` and `tree` are the C structs), t over pure terms, e over C expressions and *block* over C blocks, which may contain a *loop*. $\cdot$ is the empty spec. The specs inside a wand are wand-free, and a wand does not occur in $\textit{spec}_1$ of a *function_spec*. The additions for wands are in purple: the wand construct and the rule that every take of a wand is pinned by an equation $\textit{id} == t$.

```
struct sllist {
  int head;
  struct sllist* tail;
};

/*@
datatype List {
  Nil {},
  Cons {i32 Head, datatype List Tail}
}

predicate [rec] (datatype List) Lrep(pointer p) {
  if (is_null(p)) {
    return Nil{};
  } else {
    take H = RW<struct sllist>(p);
    assert (MINi32() <= H.head);
    assert (H.head <= MAXi32());
    take T = Lrep(H.tail);
    return (Cons { Head: H.head, Tail: T });
  }
}

function [rec] (datatype List) AddOne(datatype List l) {
  match l {
    Nil {} => { Nil {} }
    Cons {Head: h, Tail: t} =>
      { Cons { Head: h + 1i32, Tail: AddOne(t) } }
  }
}

function [rec] (boolean) allSafeToIncrement(datatype List l) {
  match l {
    Nil {} => { true }
    Cons {Head: h, Tail: t} =>
      { (MINi32() <= h + 1i32) && (h + 1i32 <= MAXi32())
        && allSafeToIncrement(t) }
  }
}
@*/
```

Fig. S6. The CN definitions used by the function of the addOne function of the main paper: the `List` datatype, the `Lrep` predicate, and the functions `AddOne` and `allSafeToIncrement`.

E Example Program: Execution under Fulminate

This appendix follows what Fulminate does with its environment when it runs the test of Fig. S9, from the start of `main` to the first take of the loop invariant of `insertEnd`, where the rules for the wand of the main paper take over. The program P is in Fig. S7 and Fig. S8: the structure `sllist`, the `List` datatype, the predicate `Listrep`, the pure function `insEnd`, and the functions `allocateNode` and `insertEnd`, with the loop invariant of the main paper. The test in Fig. S9 defines the list $[1, 2, 3]$ and calls `insertEnd` on it with the value 4. Table S1 lists the configuration at each of the points below.

```
struct sllist {
  int head;
  struct sllist* tail;
};

/*@
datatype List {
  Nil {},
  Cons {i32 Head, datatype List Tail}
}

predicate [rec] (datatype List) Listrep(pointer p) {
  if (is_null(p)) {
    return Nil{};
  } else {
    take H = RW<struct sllist>(p);
    take T = Listrep(H.tail);
    return (Cons { Head: H.head, Tail: T });
  }
}

function [rec] (datatype List) insEnd(datatype List l, i32 value) {
  match l {
    Nil {} => { Cons { Head: value, Tail: Nil {}} }
    Cons {Head: h, Tail: t} => { Cons { Head: h, Tail: insEnd(t, value) } }
  }
}
@*/
```

Fig. S7. The definitions of the running example of the main paper, in the program P: the structure `sllist`, the `List` datatype, the predicate `Listrep` and the pure function `insEnd`.

The configuration. Fulminate runs a function and its specification as a sequence of steps of the form $P, E_g, H, incdec, sd \vdash \langle spec, V, R_1, R_2, G\rangle \longrightarrow \langle spec', V', R_1', R_2', G'\rangle$. The program P gives the definition of every function, predicate and pure function by its name, so that P(`main`) is the body of `main`, P(`allocateNode`) is the body of `allocateNode`, P(`insertEnd`) the body of `insertEnd`, P(`Listrep`) is the body of the predicate. E_g maps every global variable to its address. H is the heap, a map from addresses to values, which a specification only reads. sd is the stack depth of the function whose code or specification runs, and $incdec$ says whether ownership moves up the stack (in a `requires`) or down (in an `ensures`). V maps the identifiers of the specification to values, and G maps every cell to the stack depth that owns it. The code of a function has a single set R of the cells that the program owns. Only when a specification runs is this set split in two: R_1 is the set of cells that the specification has used so far, and R_2 the set of cells that it owns but has not used yet.

```
void *cn_malloc(unsigned long size);
void cn_free_sized(void*, size_t len);

static struct sllist* allocateNode(int value)
/*@ ensures take Ret = Listrep(return);
            Ret == Cons {Head: value, Tail: Nil {}};
 @*/
{
    struct sllist* new_node = cn_malloc(sizeof(struct sllist));
    new_node->head = value;
    new_node->tail = NULL;
    return new_node;
}

struct sllist* insertEnd(struct sllist* h, int value)
/*@ requires take Lhead = Listrep(h);
    ensures take L = Listrep(return);
            L == insEnd(Lhead, value);
 @*/
{
    if (h == NULL) {
        return allocateNode(value);
    }
    struct sllist *curr = h;
    while (curr->tail != NULL)
    /*@ inv take Lcurr = Listrep(curr);
        {h} unchanged;
        wand(take Lcurr1 = Listrep(curr); Lcurr1 == insEnd(Lcurr, value);,
             take Lh = Listrep(h); Lh == insEnd(Lhead, value););
    @*/
    {
        curr = curr->tail;
    }
    curr->tail = allocateNode(value);
    return h;
}
```

Fig. S8. The functions of the program P of the running example: `allocateNode` and `insertEnd`, with its contract and the loop invariant of the main paper.

```
int main(void)
{
    struct sllist *a3 = cn_malloc(sizeof(struct sllist));
    a3->head = 3;  a3->tail = NULL;
    struct sllist *a2 = cn_malloc(sizeof(struct sllist));
    a2->head = 2;  a2->tail = a3;
    struct sllist *a1 = cn_malloc(sizeof(struct sllist));
    a1->head = 1;  a1->tail = a2;
    struct sllist *r = insertEnd(a1, 4);
    return 0;
}
```

Fig. S9. The test of `insertEnd` that the appendix follows: `main` defines the list $[1, 2, 3]$ and calls `insertEnd` on it with the value 4.

Initial configuration. The program has no global variables, so E_g is empty; the heap H is empty, since nothing has been allocated; and V, R and G are empty. Execution starts at P(`main`), with `main` at the stack depth $sd = 1$ (row 1 of Table S1).

Defining the list in `main`. The first call of `cn_malloc` returns a fresh address a_3, and the heap is extended with a cell at it: $a_3 \mapsto (3, nil)$ after the two assignments. Fulminate records that the cell is owned by the function that allocated it, $G(a_3) = 1$, and adds a_3 to R, the set of cells that `main` owns. The allocations of a_2 and a_1 do the same, with $a_2 \mapsto (2, a_3)$ and $a_1 \mapsto (1, a_2)$. After the three, $H = \{a_1 \mapsto (1, a_2), a_2 \mapsto (2, a_3), a_3 \mapsto (3, nil)\}$, $R = \{a_1, a_2, a_3\}$ and $G(a_1) = G(a_2) = G(a_3) = 1$ (row 2). There is only the one set R at this level: the code of `main` does not distinguish used and unused cells.

Calling `insertEnd`. The call `insertEnd(a1, 4)` pushes a frame for the callee, so *sd* becomes 2, and runs the `requires` part of its contract with *incdec* set to up, since the callee takes the ownership of the caller's cells. As the specification starts, R is split into the two sets of the specification level: no cell has been used, so $R_1 = \emptyset$, and all of R goes to the unused cells, $R_2 = R = \{a_1, a_2, a_3\}$ (row 3). The parameters are bound, $V = \{h \mapsto a_1, value \mapsto 4\}$, and the condition `take` $Lhead =$ `Listrep`(h) is evaluated by calling the predicate. The call pushes a map for its parameter, $p \mapsto a_1$, and steps into P(`Listrep`). As a_1 is not null, the `else` branch runs `take` $H =$ `RW`$\langle$`struct sllist`$\rangle(p)$. The primitive `RW` takes the cell a_1 out of the unused cells R_2 and into the used cells R_1, checks that the cell is owned by the caller, $G(a_1) = sd - 1 = 1$, retags it as owned by the callee, $G(a_1) = 2$, and returns its contents $(1, a_2)$, which bind H. The next take, `Listrep`$(H.tail)$, repeats this on a_2 and on a_3, with the maps $p \mapsto a_2$ and $p \mapsto a_3$ on V, and finally reaches *nil*, where the predicate returns `Nil` without taking a cell. The returns then pop the maps one by one and build the value: `Cons`$(3,$ `Nil`$) = [3]$ for the call on a_3, $[2, 3]$ for the call on a_2, and $[1, 2, 3]$ for the call on a_1, which binds *Lhead* in V. At the end of the `requires`, $R_1 = \{a_1, a_2, a_3\}$, $R_2 = \emptyset$, and all three cells are owned at the depth of `insertEnd`, $G(a_1) = G(a_2) = G(a_3) = sd = 2$ (row 4). The heap has not changed.

Running to the loop. The body of `insertEnd` is ordinary C, which does not change the specification environment. The test h == `NULL` fails, and `curr` is set to $h = a_1$. The loop head is reached once for every value of `curr`, a_1, a_2 and a_3, and the invariant is evaluated at each visit, with the C variables that it mentions, here `curr`, added to V. The loop body only moves `curr` along the list, so the heap and the ownership at the start of each visit are those of row 4. We follow the visit with $curr = a_3$, the one of the main paper. Its configuration has $V = \{h \mapsto a_1, value \mapsto 4, Lhead \mapsto [1, 2, 3], curr \mapsto a_3\}$.

Evaluating the invariant. Fulminate evaluates a loop invariant like any other specification, but with the ownership of the cells that the invariant visits lowered by one, to $sd - 1$, while it runs, so that a cell that two conditions of the invariant claim is detected, as the second claim finds it already lowered. The first condition, `take` $Lcurr =$ `Listrep`$(curr)$, calls the predicate on a_3. The primitive `RW` finds a_3 among the cells that the function has used and checks that it is owned at the depth of the function, $G(a_3) = 2$, lowers it, $G(a_3) = 1$, and returns $(3, nil)$. The call on *nil* returns `Nil` and the call on a_3 returns $[3]$, which binds *Lcurr* (row 5). The cells a_1 and a_2 are not visited by this condition, so they keep the depth 2. The constraint $\{h\}$ `unchanged` then checks that the value of h is the one it had at the entry of the function.

From Fulminate to the wand. The next condition of the invariant is the wand, which Fulminate does not have. At this point the configuration is the one of row 5: V as above with $Lcurr \mapsto [3]$, $R_1 = \{a_1, a_2, a_3\}$, $R_2 = \emptyset$, and $G = \{a_1 \mapsto 2, a_2 \mapsto 2, a_3 \mapsto 1\}$. The rules for the wand of the main paper start from it. They run the antecedent from the unused cells $R_{2pos} = R_1$ with an empty R_{1pos}, then run the consequent, which lowers the cells of the footprint, a_1 and a_2, to $sd - 1$. The wand leaves V, R_1 and R_2 as they were and only changes G, to $G' = \{a_1 \mapsto 1, a_2 \mapsto 1, a_3 \mapsto 1\}$ (row 6). When the invariant has been evaluated, Fulminate raises the cells that it lowered back to the

depth of the function, $G(a_1) = G(a_2) = G(a_3) = 2$ (row 7), and the loop continues. The invariant is then evaluated again, or, after the last visit, the condition of the loop fails, and the statement after the loop, `curr->tail = allocateNode(value)`, runs. The call of `allocateNode` behaves like the call of `insertEnd`: its own contract hands the new cell, owned at the depth of `allocateNode`, to `insertEnd`, one level down the stack. The `ensures` part of `insertEnd` is evaluated in the same way as its `requires` part, with *incdec* set to down, and hands the cells of the returned list back to `main`.

Table S1. The configuration of Fulminate at the points of the test of Fig. S9 that the appendix follows. *sd* is the stack depth, V shows only the bindings that were added since the previous row, and $G(a_1, a_2, a_3)$ lists the stack depth that owns each cell. In the code of `main` (rows 1 and 2) there is the single set R; when a specification starts (row 3), R is split into R_1 and R_2. The heap H, once the list is built, is $a_1 \mapsto (1, a_2)$, $a_2 \mapsto (2, a_3)$, $a_3 \mapsto (3, nil)$ and does not change; $E_g = \emptyset$ throughout.

	Point	*sd*	V (added)	R_1	R_2	$G(a_1, a_2, a_3)$
1	start of `main`	1	$\emptyset$	$R = \emptyset$		–
2	list built in `main`	1	$\emptyset$	$R = \{a_1, a_2, a_3\}$		$(1, 1, 1)$
3	call of `insertEnd`, specification starts	2	$h \mapsto a_1$, *value* $\mapsto 4$	$\emptyset$	$\{a_1, a_2, a_3\}$	$(1, 1, 1)$
4	`requires` of `insertEnd` done	2	*Lhead* $\mapsto [1, 2, 3]$	$\{a_1, a_2, a_3\}$	$\emptyset$	$(2, 2, 2)$
5	loop head, $curr = a_3$, first take	2	$curr \mapsto a_3$, *Lcurr* $\mapsto [3]$	$\{a_1, a_2, a_3\}$	$\emptyset$	$(2, 2, 1)$
6	after the wand	2	–	$\{a_1, a_2, a_3\}$	$\emptyset$	$(1, 1, 1)$
7	end of the invariant	2	–	$\{a_1, a_2, a_3\}$	$\emptyset$	$(2, 2, 2)$

F Benchmarks and Additional Experiments

This appendix first describes the benchmarks. It then reports the time taken by the testing algorithm, the per-seed results of Opus, Sonnet, Kimi, DeepSeek and Haiku, including the token usage and the cost, the runs of Haiku with a larger number of re-prompts, and an analysis of the invariants that the testing algorithm rejected.

F.1 Benchmarks

Table S2 lists the benchmarks, the data structure each of them operates on and what it does.

F.2 Testing Time

Table S3 gives the time that the testing algorithm takes on each benchmark for test cases whose data structure has 1, 10, 100 and 1000 nodes. The times for the sizes 1 and 10 are dominated by the compilation of the test (about a quarter of a second), the time grows with the size, steeply for the lists and much more slowly for the trees. This result can be visualized in Fig. S10, which plots the average results for each data type.

F.3 Per-Seed Results

Opus. Table S4 shows that Opus solves all 27 benchmarks in each of its three seeds. Most benchmarks are solved in the first trial. The exceptions are `copyList`, `repeatInsertEnd` and `dllCopy`, which need two trials in every seed, one of the loops of `addListCount`, `addListThroughiDll` and `bstDelete` in some of the seeds, and `bstDeleteMin`, which needs up to four trials. An average seed takes about 53 minutes and costs $11.23, the highest of the five models, as Opus is the most expensive model per token.

Table S2. The 27 benchmarks, grouped by the data structure that they operate on as in the tables of the main paper (16 singly linked list, 3 doubly linked list and 8 binary search tree benchmarks), and what each of them does. Benchmarks marked "two loops" have two loops, which are tested one after the other.

Benchmark	Description
Singly linked list	
`addConditional`	Add 1 to every element that is greater than n and add 2 to every other element.
`addInRange`	Add 1 to the elements at the positions n to m of the list.
`addListCount`	Add 1 to every element, *count* times (two nested loops).
`addListThroughi`	Add 1 to the first *count* elements.
`addOne`	Add 1 to every element.
`addTwo`	Add 1 to every element, then add 1 to every element again (two loops).
`appendList`	Append the second list to the end of the first.
`checkTwoLists`	Check whether two lists of the same length are equal.
`copyList`	Copy a list into a new list.
`copyListFilter`	Copy the elements of a list that satisfy a filter into a new list.
`deleteEndleast2`	Delete the last node of a list of at least two nodes.
`findList`	Check whether a value is a member of the list.
`insertEnd`	Insert a value at the end of the list (the running example).
`repeatInsertEnd`	Build a list of n elements by inserting at the end n times.
`returnSumNode`	Add 1 to the n-th element and return a pointer to that node.
`reverseList`	Reverse a list in place.
Doubly linked list	
`addListThroughiDll`	Add 1 to the first *count* elements of a doubly linked list (two loops).
`dllAddOneWand`	Add 1 to every element of a doubly linked list.
`dllCopy`	Copy a doubly linked list into a new list.
Binary search tree	
`bstAddRight`	Add 1 to the keys on the right spine of the tree.
`bstDelete`	Delete the node with key k from a tree of at least two nodes (two loops).
`bstDeleteMin`	Delete the node with the smallest key.
`bstFind`	Check whether a key is in the tree.
`bstReturnNode`	Return a pointer to the node with the smallest key.
`findMinKey`	Return the smallest key of the tree.
`findMinValue`	Return the value of the node with the smallest key.
`insertBSTNoCheck`	Insert a key and a value into the tree.

Sonnet. Table S5 shows that Sonnet solves 26 of the 27 benchmarks in each of its three seeds; the one it never solves is `bstDelete`. Almost every benchmark is solved in the first trial, and the others need two or three trials (`dllCopy`, `bstDeleteMin`, `copyListFilter`). An average seed takes about 9 minutes and costs $3.01, the least of the models that solve this many benchmarks. The unsolved benchmark accounts for a large part of the time, as all its trials are used.

Kimi. Table S6 shows that Kimi solves 24, 27 and 25 benchmarks in its three seeds, so it varies the most between seeds: seed 41 solves everything, seed 40 misses `addListThroughiDll`, `dllCopy` and `bstDelete`, and seed 42 misses `dllCopy` and `bstDeleteMin`. Several benchmarks need many trials, such as `copyList` (11 trials in seed 40) and `bstDeleteMin` (7 trials in seed 40). The runs are long, about 2.2 hours per seed, and cost $8.41; most of this time is spent on the benchmarks with many trials and on the unsolved ones.

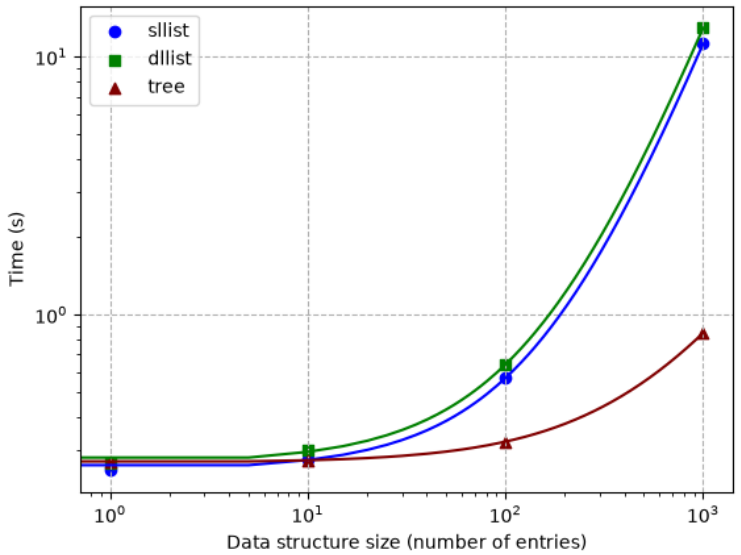
Table S3. Full performance testing results, average of ten trials for each benchmark and size. Benchmarks that take two data structures as input are labeled with ∗.

Benchmark	Time @ Size 1 (s)	Time @ Size 10 (s)	Time @ Size 100 (s)	Time @ Size 1000 (s)
addConditional	0.255	0.269	0.518	7.837
addInRange	0.250	0.281	0.569	9.562
addListCount	0.249	0.271	0.570	9.325
addListThroughi	0.252	0.279	0.573	9.412
addOne	0.248	0.270	0.531	7.829
addTwo	0.247	0.272	0.541	9.874
appendList*	0.257	0.305	0.876	26.696
checkTwoLists*	0.248	0.294	0.872	27.086
copyList	0.252	0.273	0.531	7.793
copyListFilter	0.258	0.276	0.530	7.298
deleteEndleast2$^{(2)}$	0.240	0.266	0.535	7.284
findList	0.241	0.263	0.570	9.925
insertEnd	0.245	0.268	0.522	7.253
repeatInsertEnd	0.247	0.253	0.264	13.017
returnSumNode	0.287	0.284	0.565	8.864
reverseList	0.236	0.259	0.515	10.488
sllist average	0.251	0.274	0.568	11.221
addListThroughiDll	0.269	0.302	0.677	15.336
dllAddOneWand	0.253	0.285	0.606	11.323
dllCopy	0.271	0.306	0.637	12.078
dllist average	0.264	0.298	0.640	12.912
bstAddRight	0.267	0.270	0.320	0.853
bstDelete$^{(2)}$	0.346	0.340	0.402	0.961
bstDeleteMin$^{(2)}$	0.264	0.267	0.321	0.833
bstFind	0.257	0.268	0.309	0.837
bstReturnNode	0.246	0.251	0.305	0.820
findMinKey	0.245	0.256	0.297	0.814
findMinValue	0.248	0.248	0.298	0.814
insertBSTNoCheck	0.268	0.281	0.323	0.860
tree average	0.268	0.273	0.322	0.849

DeepSeek. Table S7 shows that DeepSeek solves 26 of the 27 benchmarks in each of its three seeds, and the one that it never solves is, as for Sonnet, `bstDelete`. Most benchmarks are solved within one or two trials, and the hardest ones need up to nine (`dllCopy`), five (`repeatInsertEnd`) and four (`copyList`). DeepSeek is by far the slowest of the models, about 5.3 hours per seed, since it produces by far the most output tokens, but its cost, $7.97 per seed, is below that of Opus and Kimi.

Haiku. Table S8 shows that Haiku solves 17, 16 and 15 benchmarks in its three seeds, the fewest of the five models. The same ten benchmarks are not solved in any seed: `addInRange`, `addListCount`, `appendList`, `copyList`, `copyListFilter`, `repeatInsertEnd`, `reverseList`, `addListThroughiDll`, `dllCopy` and `bstDelete`. In addition, seed 58 misses `addTwo`, and seed 59 misses `returnSumNode` and `bstDeleteMin`. The benchmarks that Haiku solves are mostly solved in one to four trials, and a seed costs only $3.67 and takes about 13 minutes, but the benchmarks that stay unsolved use all their trials.

Fig. S10. The average performance results for each data type at each size, shown here along a quadratic curve of best fit. The gap between the sllist and dllist curve may be a result of the slight increase in overhead that comes with managing the pointer to the previous node. Performance times for trees exhibit slower growth relative to data structure size.



F.4 Haiku with More Re-prompts

Table S9 gives the trials of Haiku (seed 58) when the number of re-prompts per loop is raised from 10 to 20 and 40.

For 10 re-prompts, this seed is able to pass 16 benchmarks - one less than the best seed. 3 extra benchmarks - `addListCount`, `appendList` and `reverseList` were successfully tested with both 20 and 40 re-prompts. 40 re-prompts did not solve any additional benchmark. The increased re-prompts also did not fail any benchmark that was previously passing with 10. The number of semantic errors reported by the testing algorithm are 58 and 152 for all benchmarks corresponding to the 20 and 40 re-prompt cases. We note that the extra solved benchmarks didn't necessarily require more than 10 re-prompts. Example, `addListCount` failed for the second loop on the 10 re-prompts configuration. The 20 re-prompt configuration has the second loop passing in the 19th try, whereas the 40 re-prompt configuration passes the second loop in the first try. It takes 16 attempts for the first loop, which was solved in 3 attempts for the base attempt. We observe similar behaviour across successfully tested benchmark(s) in the base case. Example, `bstDeleteMin` required 8 attempts for the base case, and 13 for the 20 re-prompt case. This can be attributed to the inherent randomness of LLMs. Manual analysis of the LLM's output lead us to conclude that they can be stubborn and repeatedly output the same incorrect invariant, but also learn from the past mistakes and eventually tend to converge to the correct one with more re-prompts.

Table S4. Opus, seeds 35, 36, 37 (Seed 1–3), at most 10 reprompts per loop. Trials = 1+reprompts (at most 11). For benchmarks with a single loop the entry is the number of trials; for benchmarks with two loops it is a tuple (trials for loop 1, trials for loop 2). Total time, testing time, tokens (input/output) and cost are averaged per seed. Cost at $5/$25 per million input/output tokens.

Benchmark	Seed 1	Seed 2	Seed 3	Total Time (s)	Testing Time (s)	Avg. Tokens (in/out)	Avg. Cost ($)
addConditional	1	1	1	6	0.7	24.0k/0.3k	0.13
addInRange	1	1	1	28	1.0	24.3k/2.3k	0.18
addListCount	(1, 2)	(1, 1)	(1, 2)	155	1.7	64.5k/13.6k	0.66
addListThroughi	1	1	1	15	0.7	24.1k/1.2k	0.15
addOne	1	1	1	9	0.7	24.0k/0.2k	0.12
addTwo	(1, 1)	(1, 1)	(1, 1)	76	1.3	48.4k/6.4k	0.40
appendList	1	1	1	24	0.7	23.9k/2.0k	0.17
checkTwoLists	1	1	1	37	0.7	24.0k/3.3k	0.20
copyList	2	2	2	203	1.0	48.3k/17.4k	0.68
copyListFilter	1	1	1	161	0.7	24.2k/13.8k	0.47
deleteEndleast2	1	1	1	18	0.7	23.9k/1.4k	0.16
findList	1	1	1	11	0.7	23.8k/0.8k	0.14
insertEnd	1	1	1	10	0.7	23.9k/0.6k	0.14
repeatInsertEnd	2	2	2	130	0.7	48.3k/10.7k	0.51
returnSumNode	1	1	1	27	0.7	24.3k/2.2k	0.18
reverseList	1	1	1	23	0.7	24.0k/1.8k	0.17
addListThroughiDll	(1, 1)	(1, 1)	(1, 2)	698	1.7	57.7k/59.4k	1.77
dllAddOneWand	1	1	1	5	0.7	23.9k/0.2k	0.13
dllCopy	2	2	2	236	1.0	48.8k/20.1k	0.75
bstAddRight	1	1	1	7	2.3	23.8k/0.3k	0.13
bstDelete	(2, 1)	(1, 1)	(2, 1)	627	14.0	70.8k/53.4k	1.69
bstDeleteMin	4	1	2	582	3.7	58.2k/50.5k	1.55
bstFind	1	1	1	16	2.3	24.0k/0.9k	0.14
bstReturnNode	1	1	1	10	2.3	24.0k/0.5k	0.13
findMinKey	1	1	1	17	3.0	24.0k/1.3k	0.15
findMinValue	1	1	1	15	3.0	24.0k/1.1k	0.15
insertBSTNoCheck	1	1	1	30	2.3	24.3k/2.3k	0.18

F.5 Analysis of the Rejected Invariants

Table S10 classifies the invariants that the testing algorithm rejected, for every seed, before the first passing attempt of a loop. In total it rejects 475 invariants across the 15 seeds and the two longer runs of Haiku, and 344 of them (72%) are rejected because a value of the invariant differs from the value that the program computes, such as a wrong antecedent or a wrong equation of the wand; 60 are rejected because of the footprint of the wand, and 71 because of the loop ownership check or an assertion. The rejections are unevenly spread over the models: Opus has only 3, Sonnet 29, DeepSeek 16 and Kimi 61, while Haiku has 156 in its three seeds, and the longer runs of Haiku add 58 and 152 rejections without solving more benchmarks than the three seeds do. The test cases on which the invariants fail are small, with at most 12 cells and an average of 2.13 cells, so the testing algorithm finds the semantic errors with very small tests, and the cost of a test is small.

Table S5. Sonnet, seeds 46, 47, 48 (Seed 1–3), at most 10 reprompts per loop. Trials = 1+reprompts (at most 11). For benchmarks with a single loop the entry is the number of trials (or × if the loop stayed unresolved); for benchmarks with two loops it is a tuple (trials for loop 1, trials for loop 2), where × marks a loop that stayed unresolved and N/A a second loop that was not reached because loop 1 failed. Total time, testing time, tokens (input/output) and cost are averaged per seed. Cost at $2/$10 per million input/output tokens.

Benchmark	Seed 1	Seed 2	Seed 3	Total Time (s)	Testing Time (s)	Avg. Tokens (in/out)	Avg. Cost ($)
addConditional	1	1	1	3	1.0	24.0k/0.2k	0.05
addInRange	1	1	1	3	1.0	24.3k/0.3k	0.05
addListCount	(1, 5)	(1, 3)	(1, 1)	39	1.7	97.7k/4.1k	0.24
addListThroughi	1	1	1	3	0.7	24.1k/0.2k	0.05
addOne	1	1	1	3	0.7	24.0k/0.2k	0.05
addTwo	(1, 1)	(1, 1)	(1, 1)	9	1.3	48.4k/0.9k	0.11
appendList	1	1	1	5	0.7	23.9k/0.5k	0.05
checkTwoLists	1	1	1	5	0.7	24.0k/0.4k	0.05
copyList	1	1	1	8	0.7	24.0k/0.9k	0.06
copyListFilter	1	1	2	17	1.0	33.1k/2.0k	0.09
deleteEndleast2	1	2	1	6	0.7	32.0k/0.6k	0.07
findList	1	1	1	3	0.7	23.9k/0.2k	0.05
insertEnd	1	1	1	5	0.7	23.9k/0.3k	0.05
repeatInsertEnd	1	1	1	8	0.7	24.0k/0.8k	0.06
returnSumNode	1	1	1	3	0.7	24.3k/0.2k	0.05
reverseList	1	1	1	2	0.7	24.0k/0.1k	0.05
addListThroughiDll	(2, 1)	(1, 7)	(1, 1)	110	2.3	109.2k/12.9k	0.35
dllAddOneWand	1	1	1	3	0.7	23.9k/0.1k	0.05
dllCopy	3	3	2	37	1.0	65.2k/4.3k	0.17
bstAddRight	1	1	1	4	2.3	23.8k/0.1k	0.05
bstDelete	(×, N/A)	(×, N/A)	(×, N/A)	230	37.0	323.5k/23.2k	0.88
bstDeleteMin	2	3	1	26	3.7	49.4k/2.8k	0.13
bstFind	1	1	1	4	2.3	24.0k/0.1k	0.05
bstReturnNode	1	1	1	4	2.3	24.0k/0.1k	0.05
findMinKey	1	1	1	5	2.7	24.0k/0.1k	0.05
findMinValue	1	1	1	6	2.7	24.0k/0.2k	0.05
insertBSTNoCheck	1	1	1	5	2.3	24.3k/0.1k	0.05

Table S6. Kimi, seeds 40, 41, 42 (Seed 1–3), at most 10 reprompts per loop. Trials = 1+reprompts (at most 11). For benchmarks with a single loop the entry is the number of trials (or × if the loop stayed unresolved); for benchmarks with two loops it is a tuple (trials for loop 1, trials for loop 2), where × marks a loop that stayed unresolved and N/A a second loop that was not reached because loop 1 failed. Total time, testing time, tokens (input/output) and cost are averaged per seed. Cost at $3/$15 per million input/output tokens.

Benchmark	Seed 1	Seed 2	Seed 3	Total Time (s)	Testing Time (s)	Avg. Tokens (in/out)	Avg. Cost ($)
addConditional	1	1	1	7	0.7	16.5k/0.1k	0.05
addInRange	4	2	2	67	2.0	46.0k/2.6k	0.18
addListCount	(1, 2)	(1, 3)	(1, 3)	119	1.7	61.0k/4.3k	0.25
addListThroughi	1	1	1	8	0.7	16.5k/0.2k	0.05
addOne	1	1	1	7	0.7	16.5k/0.2k	0.05
addTwo	(1, 1)	(1, 1)	(1, 1)	41	1.3	33.1k/1.3k	0.12
appendList	5	2	1	53	2.0	45.2k/1.5k	0.16
checkTwoLists	1	1	1	18	0.7	16.4k/0.6k	0.06
copyList	11	5	3	917	2.7	111.3k/36.4k	0.88
copyListFilter	2	2	2	190	0.7	33.3k/7.1k	0.21
deleteEndleast2	1	1	1	13	0.7	16.4k/0.3k	0.05
findList	1	1	1	8	0.7	16.4k/0.2k	0.05
insertEnd	1	1	1	10	0.7	16.4k/0.3k	0.05
repeatInsertEnd	3	4	7	764	1.3	78.3k/28.9k	0.67
returnSumNode	2	1	2	61	1.0	28.9k/2.4k	0.12
reverseList	1	1	4	34	1.0	33.4k/1.3k	0.12
addListThroughiDll	(2, ×)	(1, 10)	(1, 4)	1,680	6.0	171.6k/63.8k	1.47
dllAddOneWand	1	1	1	6	0.7	16.4k/0.1k	0.05
dllCopy	×	5	×	1,534	2.7	159.2k/60.3k	1.38
bstAddRight	1	1	1	7	2.3	16.4k/0.1k	0.05
bstDelete	(×, N/A)	(1, 2)	(6, 3)	1,005	26.7	149.4k/34.9k	0.97
bstDeleteMin	7	5	×	1,482	5.3	134.8k/51.3k	1.17
bstFind	1	1	1	8	2.3	16.5k/0.2k	0.05
bstReturnNode	1	1	1	10	2.3	16.4k/0.2k	0.05
findMinKey	1	1	1	10	2.7	16.4k/0.2k	0.05
findMinValue	1	1	1	10	2.7	16.4k/0.1k	0.05
insertBSTNoCheck	1	1	1	10	2.7	16.7k/0.2k	0.05

Table S7. DeepSeek, seeds 66, 67, 68 (Seed 1–3), at most 10 reprompts per loop. Trials = 1+reprompts (at most 11). For benchmarks with a single loop the entry is the number of trials (or × if the loop stayed unresolved); for benchmarks with two loops it is a tuple (trials for loop 1, trials for loop 2), where × marks a loop that stayed unresolved and N/A a second loop that was not reached because loop 1 failed. Total time, testing time, tokens (input/output) and cost are averaged per seed. Cost at $1.32/$3.96 (DeepSeek-V4-Pro peak list price, cache-miss input) per million input/output tokens.

Benchmark	Seed 1	Seed 2	Seed 3	Total Time (s)	Testing Time (s)	Avg. Tokens (in/out)	Avg. Cost ($)
addConditional	1	1	1	43	1.0	17.1k/3.5k	0.04
addInRange	1	1	1	146	1.0	17.3k/13.6k	0.08
addListCount	(1, 1)	(1, 3)	(1, 1)	955	1.7	45.9k/84.2k	0.39
addListThroughi	1	1	2	285	0.7	23.0k/22.9k	0.12
addOne	1	1	1	53	0.7	17.1k/4.2k	0.04
addTwo	(1, 2)	(1, 1)	(1, 1)	757	1.3	40.2k/62.6k	0.30
appendList	1	2	1	205	1.0	22.8k/17.6k	0.10
checkTwoLists	1	1	1	256	0.7	17.1k/23.3k	0.11
copyList	3	2	4	1,404	1.3	51.8k/126.7k	0.57
copyListFilter	1	2	1	525	0.7	23.0k/47.0k	0.22
deleteEndleast2	1	1	1	178	0.7	17.0k/15.4k	0.08
findList	1	1	1	41	0.7	17.0k/3.6k	0.04
insertEnd	2	1	1	163	0.7	22.8k/13.9k	0.09
repeatInsertEnd	3	5	3	1,402	1.3	63.5k/126.5k	0.58
returnSumNode	1	1	1	201	0.7	17.3k/17.5k	0.09
reverseList	1	1	1	275	0.7	17.1k/23.1k	0.11
addListThroughiDll	(1, 3)	(1, 4)	(1, 3)	2,299	2.3	77.3k/194.0k	0.87
dllAddOneWand	1	2	1	21	0.7	22.8k/1.8k	0.04
dllCopy	9	4	6	2,595	2.3	113.8k/235.4k	1.08
bstAddRight	2	2	2	88	2.3	34.2k/6.8k	0.07
bstDelete	(×, N/A)	(×, N/A)	(×, N/A)	5,819	17.7	220.5k/493.0k	2.24
bstDeleteMin	2	2	2	956	3.3	34.8k/89.2k	0.40
bstFind	1	1	1	45	2.3	17.1k/3.6k	0.04
bstReturnNode	1	1	1	72	2.3	17.0k/6.0k	0.05
findMinKey	1	1	1	153	3.0	17.1k/13.7k	0.08
findMinValue	1	1	1	147	3.0	17.1k/13.5k	0.08
insertBSTNoCheck	1	1	1	112	2.3	17.3k/8.9k	0.06

Table S8. Haiku, seeds 57, 58, 59 (Seed 1–3), at most 10 reprompts per loop. Trials = 1+reprompts (at most 11). For benchmarks with a single loop the entry is the number of trials (or × if the loop stayed unresolved); for benchmarks with two loops it is a tuple (trials for loop 1, trials for loop 2), where × marks a loop that stayed unresolved and N/A a second loop that was not reached because loop 1 failed. Total time, testing time, tokens (input/output) and cost are averaged per seed. Cost at $1/$5 per million input/output tokens.

Benchmark	Seed 1	Seed 2	Seed 3	Total Time (s)	Testing Time (s)	Avg. Tokens (in/out)	Avg. Cost ($)
addConditional	1	1	1	3	1.0	19.0k/0.2k	0.02
addInRange	×	×	×	78	5.0	226.8k/7.5k	0.26
addListCount	(×, N/A)	(3, ×)	(×, N/A)	44	4.3	241.4k/3.5k	0.26
addListThroughi	1	3	1	7	1.0	32.2k/0.5k	0.03
addOne	1	1	1	3	0.7	19.0k/0.1k	0.02
addTwo	(10, 1)	(×, N/A)	(10, 1)	42	5.7	221.3k/3.2k	0.24
appendList	×	×	×	51	4.3	221.1k/4.2k	0.24
checkTwoLists	4	3	1	13	1.0	51.7k/1.2k	0.06
copyList	×	×	×	47	1.3	226.5k/4.6k	0.25
copyListFilter	×	×	×	69	2.3	228.0k/7.6k	0.27
deleteEndleast2	1	1	4	9	0.7	38.2k/0.7k	0.04
findList	1	1	1	2	0.7	18.9k/0.1k	0.02
insertEnd	1	3	9	13	1.0	85.9k/0.9k	0.09
repeatInsertEnd	×	×	×	42	1.7	224.3k/3.4k	0.24
returnSumNode	5	6	×	32	2.0	150.9k/2.6k	0.16
reverseList	×	×	×	56	4.7	222.3k/4.9k	0.25
addListThroughiDll	(×, N/A)	(×, N/A)	(×, N/A)	62	4.7	227.9k/5.2k	0.25
dllAddOneWand	3	1	1	4	0.7	31.7k/0.3k	0.03
dllCopy	×	×	×	53	2.3	227.8k/4.9k	0.25
bstAddRight	1	1	1	4	2.7	18.9k/0.1k	0.02
bstDelete	(×, N/A)	(×, N/A)	(×, N/A)	63	3.7	238.4k/5.6k	0.27
bstDeleteMin	3	8	×	39	3.3	147.6k/3.4k	0.16
bstFind	6	1	1	10	3.0	51.8k/0.5k	0.05
bstReturnNode	1	2	5	10	2.7	51.5k/0.7k	0.05
findMinKey	1	1	1	5	3.0	19.0k/0.1k	0.02
findMinValue	1	1	1	5	3.0	19.0k/0.1k	0.02
insertBSTNoCheck	4	4	6	20	2.7	91.8k/1.6k	0.10

Table S9. Haiku, seed 58, with at most 10, 20 and 40 reprompts per loop (at most 11, 21 and 41 trials). For benchmarks with a single loop the entry is the number of trials (or × if it stayed unresolved); for benchmarks with two loops it is a tuple (trials for loop 1, trials for loop 2), where × marks a loop that stayed unresolved and N/A a second loop that was not reached because loop 1 failed.

Benchmark	Max 10	Max 20	Max 40
addConditional	1	1	1
addInRange	×	×	×
addListCount	(3, ×)	(6, 19)	(16, 1)
addListThroughi	3	1	1
addOne	1	1	1
addTwo	(×, N/A)	(×, N/A)	(×, N/A)
appendList	×	14	4
checkTwoLists	3	3	3
copyList	×	×	×
copyListFilter	×	×	×
deleteEndleast2	1	3	1
findList	1	1	1
insertEnd	3	1	23
repeatInsertEnd	×	×	×
returnSumNode	6	4	5
reverseList	×	3	39
addListThroughiDll	(×, N/A)	(×, N/A)	(×, N/A)
dllAddOneWand	1	1	1
dllCopy	×	×	×
bstAddRight	1	1	1
bstDelete	(×, N/A)	(×, N/A)	(×, N/A)
bstDeleteMin	8	13	5
bstFind	1	1	4
bstReturnNode	2	1	2
findMinKey	1	1	1
findMinValue	1	1	1
insertBSTNoCheck	4	1	2

Table S10. Invariants rejected by the testing algorithm, for every seed, before the first passing attempt of a loop. *Footprint* is the uncovered precondition cell, the overlap of the consequent with a resource outside the wand and the overlap of the antecedent; *Other* is the failures of the loop ownership check and of assertions. The size is the number of heap cells of the failing test case (min/max/avg).

Model	Seed	Rejected	Value mismatch	Footprint	Other	Test size
Opus	35	2	2	0	0	3/3/3.00
Opus	36	0	0	0	0	–
Opus	37	1	1	0	0	4/4/4.00
Sonnet	46	7	7	0	0	2/3/2.14
Sonnet	47	14	12	2	0	1/7/2.21
Sonnet	48	8	7	1	0	2/3/2.12
Haiku	57	48	37	4	7	0/8/1.94
Haiku	58	47	32	8	7	0/5/1.96
Haiku	59	61	42	9	10	0/3/2.16
Kimi	40	36	19	15	2	2/9/3.31
Kimi	41	10	9	1	0	1/4/2.50
Kimi	42	15	12	3	0	1/12/3.00
DeepSeek	66	5	3	0	2	2/6/2.80
DeepSeek	67	5	1	1	3	3/5/3.40
DeepSeek	68	6	4	0	2	1/3/2.50
Haiku	58, 20 re-prompts	58	46	5	7	0/7/2.21
Haiku	58, 40 re-prompts	152	110	11	31	0/7/1.69
Total		475	344	60	71	0/12/2.13